

Design Rationale

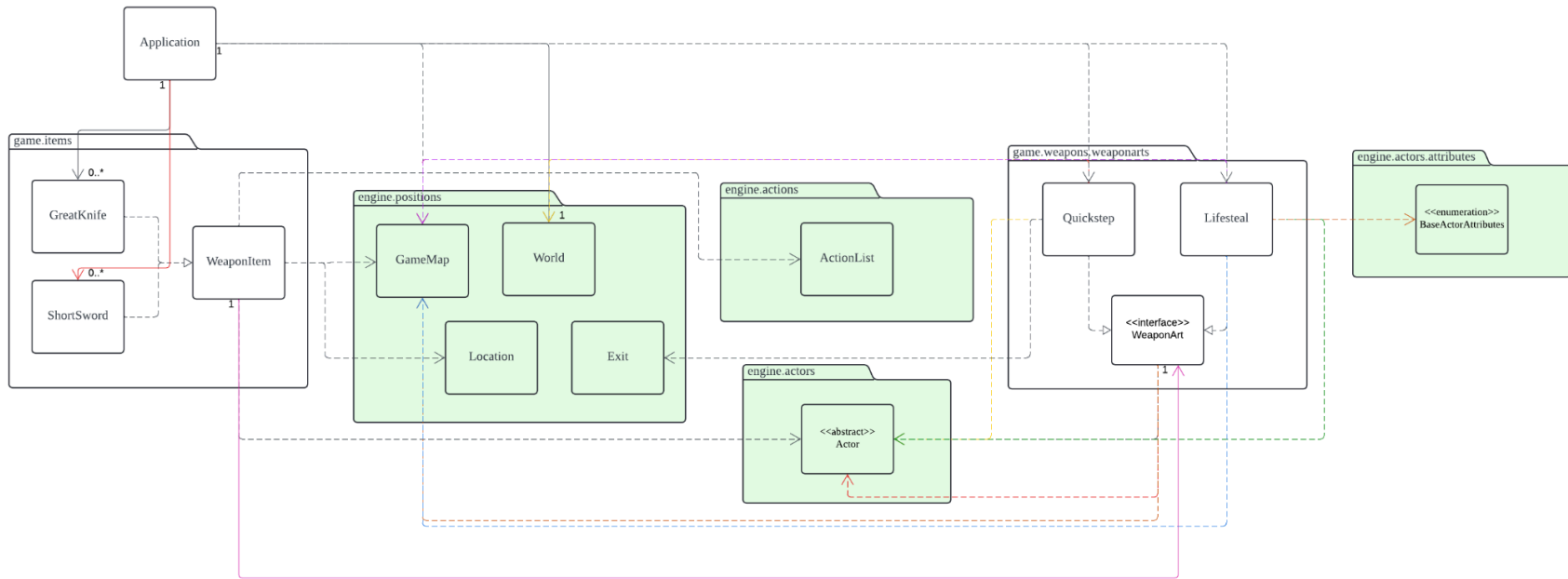
Design Goal

The design aims to break down the system into smaller, more manageable classes to promote modularity and encapsulation. In this system, each class will have a clear and distinct responsibility, making it easier to understand and maintain. By adhering to the SOLID principle and DRY principle, the system should allow for easy extension and promote reusability. This involves identifying common functionalities and abstracting them into reusable components as well as anticipating potential changes to enhance code flexibility and adaptability to changing requirements and future extensions.

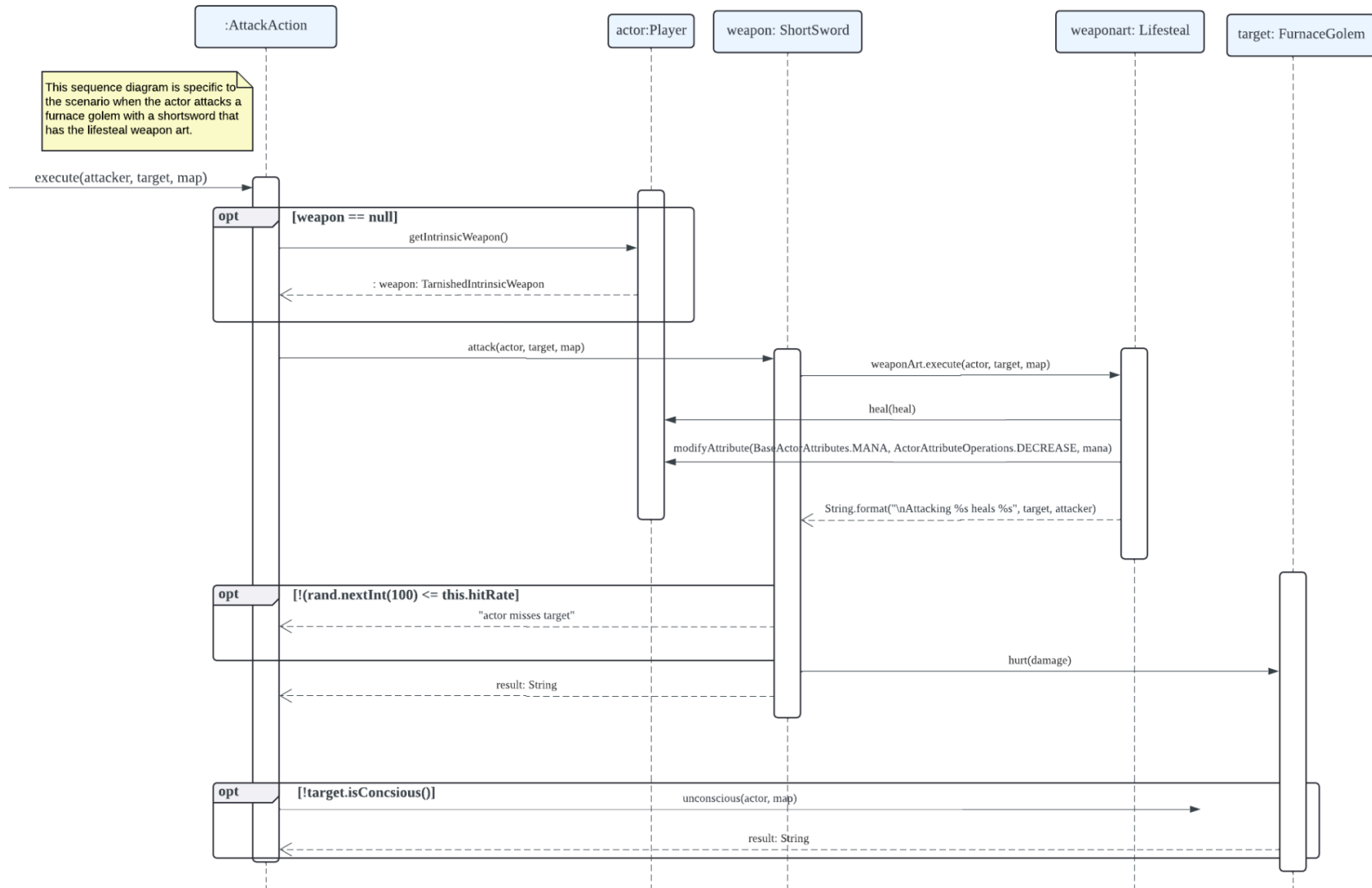
Changes made from A1

Classes Modified/Created	Roles and Responsibilities	Rationale
Enemy	An abstract class representing an enemy. Relationships: It extends Actor class to achieve actor functionality. It depends on Behaviour to create a treemap of the enemy's behaviour.	Alternate Solution: Let every enemy classes extends actor and define their own behaviours in each of their classes. Finalised solution: Create an Enemy abstract class and let any enemy classes to extends from it. Made WeaponItem an abstract class.
WeaponItem (extends Item implements Weapon)	An abstract class representing an item that can be used as a weapon. Relationships: 1. It extends Item class to achieve portable functionality. 2. It implements Weapon class to achieve attack functionality. 3. It depends on WeaponArt to add the weapon art to itself.	Reasons for Decision: 1. Every enemy classes have similar codes, enemy abstract class can be reusable to avoid code duplication. (<i>DRY Principle</i>) 2. WeaponItem class needs to be abstract to ensure it cannot be instantiated directly. It introduces an inconsistent contract due to polymorphism in OO. (<i>Liskov substitution Principle</i>)

REQ1: Weapon Arts



Req 1 UML Diagram

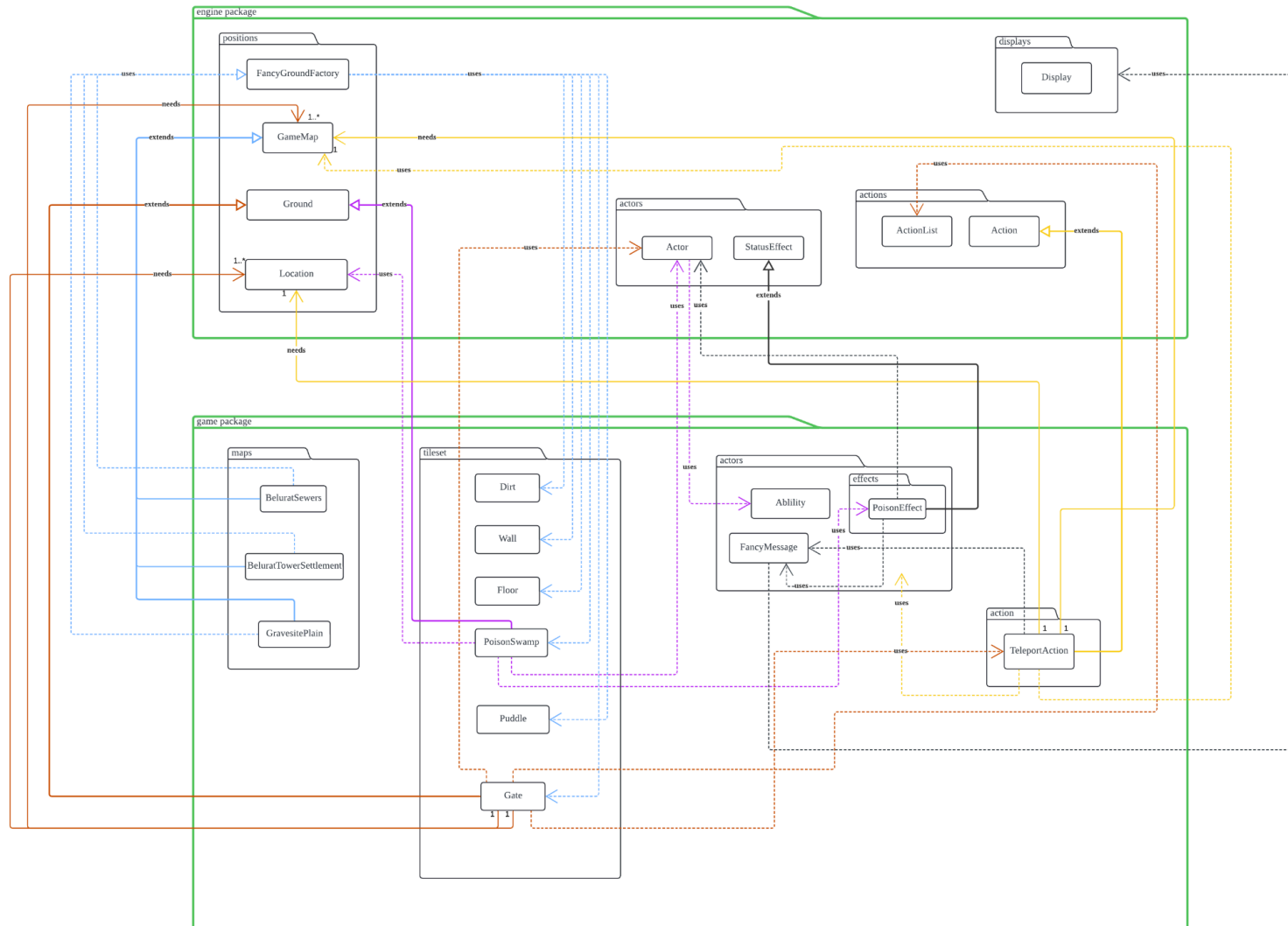


Req 1 Sequence Diagram

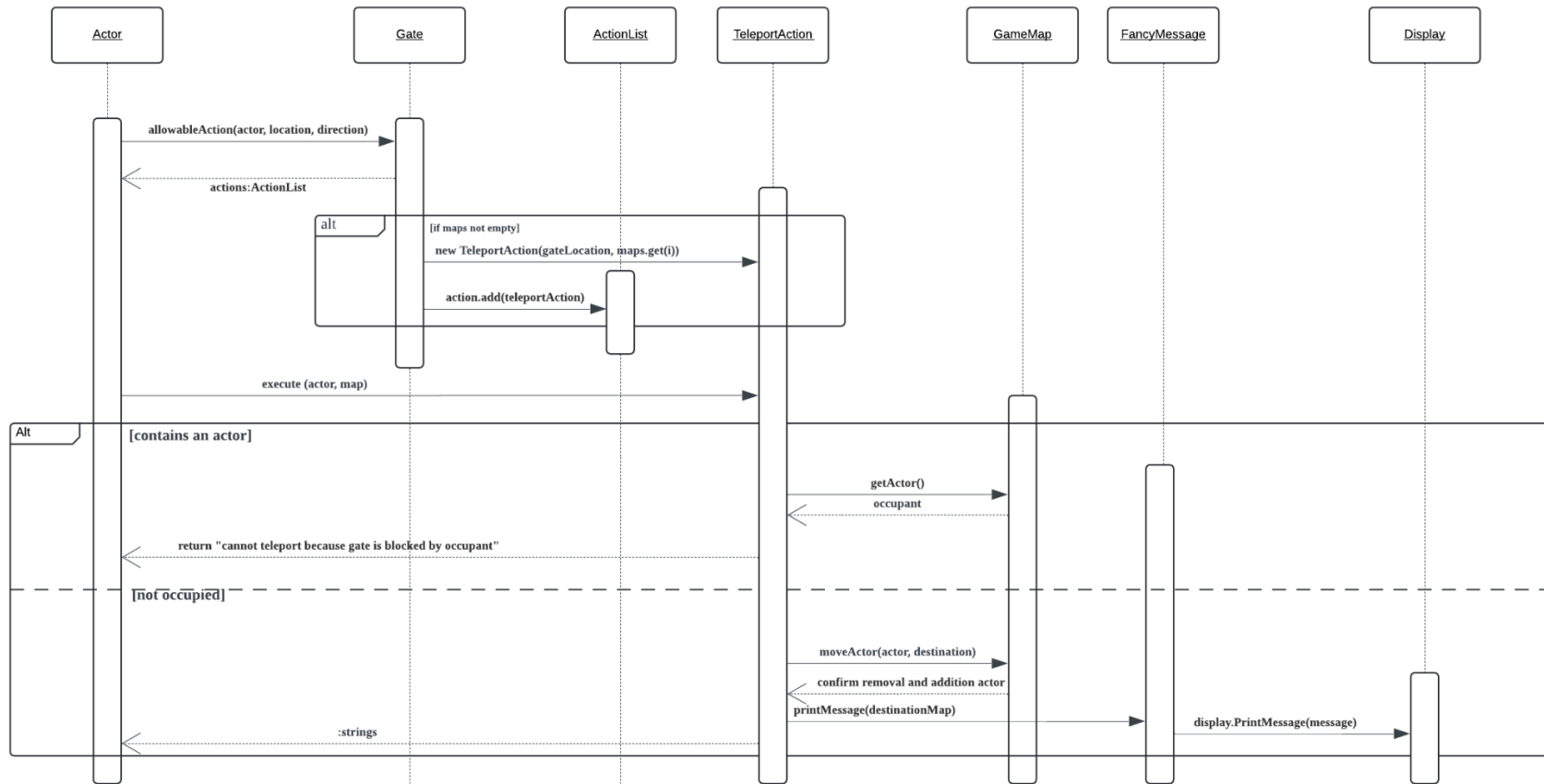
Classes Modified/Created	Roles and Responsibilities	Rationale
WeaponArt	Interface for weapon arts. Relationships: 1. It depends on Actor class to determine which actor performs the weapon art. 2. It depends on the GameMap class to retrieve the location of the actor and move the actor to a new location.	Alternate Solution: Instead of dividing each weapon art into separate classes, combine everything into a single class. Implement each of the weapon art's effects in the class and use if-else statements to determine which weapon art is to be activated. Finalised solution: Create separate classes for each of the weapon arts by implementing the WeaponArt interface. Define each of the execute functions for each weapon arts in their respective classes. Extend the WeaponItem by introducing a WeaponArt attribute which will be added to the weapons through the enchant() function. The weapon arts are to be added to the weapons inside the Application class.
Lifesteal (implements WeaponArt)	Class representing the “lifesteal” weapon art. Relationships: 1. It implements WeaponArt interface to achieve weapon art functionality. 2. It depends on BaseActorAttributes class to access the mana attribute of the player. 3. It depends on ActorAttributeOperations class to decrease the mana depending on the weapon art's mana cost.	Reasons for Decision: 1. By creating a new class for each of the weapon arts, we are ensuring that each weapon art class has only a single responsibility. This is done by implementing the behaviour of the weapon art in its respective class which improves code maintainability and extendability as any changes made to a specific weapon art will not affect other weapon arts. (<i>Single Responsibility Principle</i>) 2. The WeaponArt interface allows the code to be open for extension but closed for modification. For example, if another weapon art were to be created, a new class could be created implementing the WeaponArt interface without modifying any of the existing code. This reduces the risk of introducing new bugs and breaking the existing code. (<i>Open Closed Principle</i>) 3. Different types of weapon arts can be used interchangeably
Quickstep (implements WeaponArt)	Class representing the “quickstep” weapon art. Relationships: 1. It implements WeaponArt interface to achieve weapon art functionality.	

	- It depends on Exit class to retrieve the list of exits of the actor. - It depends on Location to get the destination for each of the player's exits.	whenever a WeaponArt is expected in the enchant() function without breaking the unexpected behaviour. (<i>Liskov substitution Principle</i>) - The WeaponItem depends on abstraction(WeaponArt). By doing so, multiple if-else statements and instanceof checks can be avoided. This means that the WeaponItem does not need to depend on every weapon arts class, making it more maintainable as it does not depend on low-level modules. (<i>Dependency Inversion Principle</i>) - Lifesteal and Quickstep depend on the WeaponArt interface, overriding the execute() function which handles the behaviour of the weapon arts and prevents it from repeating code. (<i>DRY Principle</i>)
WeaponItem (extends Item implements Weapon)	An abstract class representing an item that can be used as a weapon. Relationships: - It extends Item class to achieve portable functionality. - It implements Weapon class to achieve attack functionality. - It depends on WeaponArt to add the weapon art to itself.	Limitations and Tradeoffs: - Introducing new weapon arts means creating more classes. This can cause even greater complexities and make it difficult to get the flow of the implementation logic. - Using an interface for WeaponArt instead of an abstract class might introduce code duplication which violates the DRY principle as there might exist similar implementations across the weapon arts.
Application	Class responsible to start the game. Relationships: - It depends on GreatKnife class to instantiate GreatKnife object. - It depends on ShortSword class to instantiate ShortSword object. - It depends on the World class to basically run the whole program.	

REQ2: Belurat, Tower Settlement



Req 2 UML Diagram



Req 2 Sequence Diagram

Classes Modified/Created	Roles and Responsibilities	Rationale
BeluratSewers, BeluratTowerSettlement	Classes representing new areas: BeluratSewers and BeluratTowerSettlement. Relationships: 1. Extend GameMap to provide environments for actor interaction, terrain generation, and item placements. 2. Use FancyGroundFactory to generate consistent terrain types such as Dirt, Wall, Puddle, and PoisonSwamp.	Alternate Solution: Instead of creating separate classes for each new area (BeluratSewers and BeluratTowerSettlement), combine both into a single class and use methods to manage the unique properties and features of each environment. Similarly, poison and teleportation functionality could be integrated directly within map or actor classes, rather than separate terrain and action classes. Finalized Solution: 1. Create distinct classes for BeluratSewers and BeluratTowerSettlement, both extending GameMap. Utilize FancyGroundFactory to generate consistent terrain types across all maps, including Dirt, Wall, Puddle, and PoisonSwamp. 2. Introduce the PoisonSwamp class as a specialized terrain that extends Ground to manage poison effects on non-resistant actors. 3. Implement the Gate class to handle teleportation between different areas, using TeleportAction for consistent teleportation behavior. 4. Add DeathAction for handling actor death events, and FancyMessage to provide narrative visual feedback such as teleportation messages and map changes. Reasons for Decision: Modular Design: 1. Separation of Responsibilities: By creating distinct classes for BeluratSewers, BeluratTowerSettlement,
PoisonSwamp, PoisonEffect	Class representing hazardous terrain in BeluratSewers. Relationships: 1. Extends Ground and inflicts poison on actors lacking POISON_RES ability. 2. Integrated with the Ability enum and PoisonEffect to check resistance and apply poison.	
Gate	A terrain type extending Ground to provide teleportation functionality. Relationships: 1. Connects multiple GameMap instances, allowing seamless	

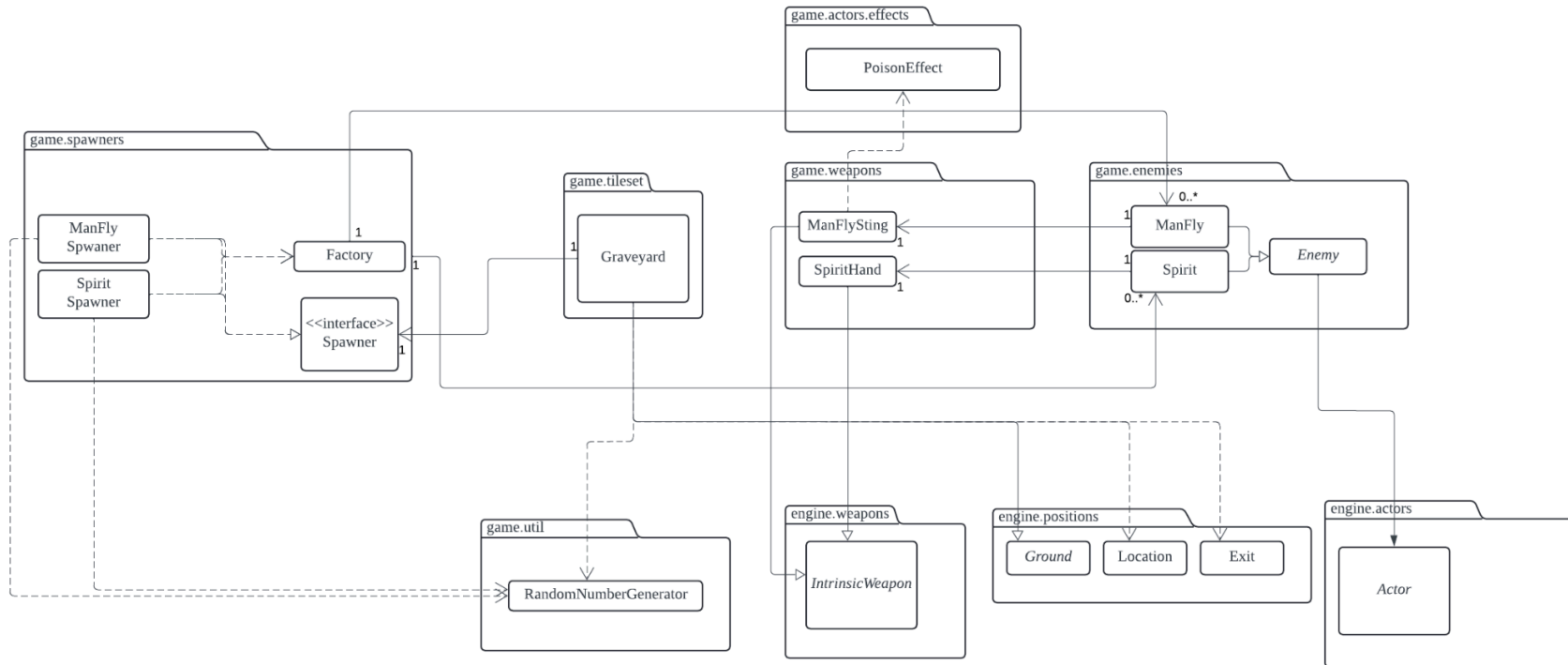
	movement between different game areas. Managed using the TeleportAction class to handle teleportation interaction with actors.	PoisonSwamp, and Gate, the solution adheres to the Single Responsibility Principle. Each class focuses on specific functionality—map management, terrain effects, and teleportation—making the design modular, easier to maintain, and extendable. - Dedicated Death Handling: The introduction of DeathAction ensures that actor death logic is isolated from the core Actor class. This helps maintain the actor's core responsibilities while providing a dedicated way to manage what happens upon death. Reusable Terrain and Actions: - Centralized Terrain Generation: Using FancyGroundFactory ensures a DRY (Don't Repeat Yourself) approach for terrain generation across all maps. This follows the Open/Closed Principle, allowing new terrain types to be added easily without modifying existing classes. - PoisonSwamp and PoisonEffect: The PoisonSwamp class, in conjunction with PoisonEffect, provides a reusable system for managing poison damage dynamically. Actors interact differently based on their Ability (e.g., POISON_RES), enhancing strategic gameplay and keeping the poison logic consistent across areas. Scalable and Dynamic Navigation: - Teleportation System: The Gate class, paired with TeleportAction, provides a scalable and consistent way for players to navigate between different maps. This modular approach ensures that any new teleportation logic can be added without affecting other game components, maintaining adherence to the Open/Closed Principle.
DeathAction	Class representing the action performed when an actor dies. Relationships: - Implements Action interface: Represents the action triggered when an actor's health reaches zero. - Depends on Actor class: Manages the actor's removal from the game once the death action is executed. - Depends on GameMap class: Utilizes GameMap to update the map by removing the actor's presence after death. - Depends on StatusEffect class: Applies any final status effects such as dropping loot or causing lasting consequences when an actor dies.	
TeleportAction	A class responsible for managing teleportation interactions between actors and gates. Relationships:	

	1. Extends the Action class, providing teleportation functionality for actors in the game world. 2. Moves actors between specified GameMap locations upon interaction with the Gate class. 3. Displays feedback to the player using FancyMessage to indicate successful teleportation to a new area.	2. Visual Feedback with FancyMessage: The integration of FancyMessage with teleportation provides clear and immersive player feedback, helping players stay oriented during transitions between maps, which enhances the overall game experience. Simplified Actor and Effect Management: 1. Actor Death Management: The DeathAction class helps manage the actor's removal from the game upon death, applying final status effects like loot drops. This reduces the complexity within the Actor class, ensuring consistency across all actors during death events. 2. Status Effect Lifecycle: The use of StatusEffect and its derivative PoisonEffect helps maintain a consistent lifecycle for poison effects, ensuring that damage is applied and removed accurately without duplication across classes. Limitations and Tradeoffs: Complexity in Maintenance: 1. Adding new maps or features involves creating more classes, which increases the complexity of the codebase. Maintaining separate classes for each area means new developers need to familiarize themselves with multiple class definitions to understand the system comprehensively. Tight Coupling with Specific Abilities: 1. The reliance on specific abilities like POISON_RES introduces tight coupling between PoisonSwamp,
PoisonEffect	A class derived from StatusEffect to inflict poison damage on actors. Relationships: 1. Continuously decreases the actor's health over multiple turns if the actor steps on a PoisonSwamp and lacks poison resistance. 2. Works in conjunction with PoisonSwamp and checks the actor's Ability to determine if they have the POISON_RES capability. 3. Ensures consistent application and removal of poison through the StatusEffect lifecycle, managing the poison's impact over time.	
Ability, POISON_RES	Ability is an enumeration that defines	

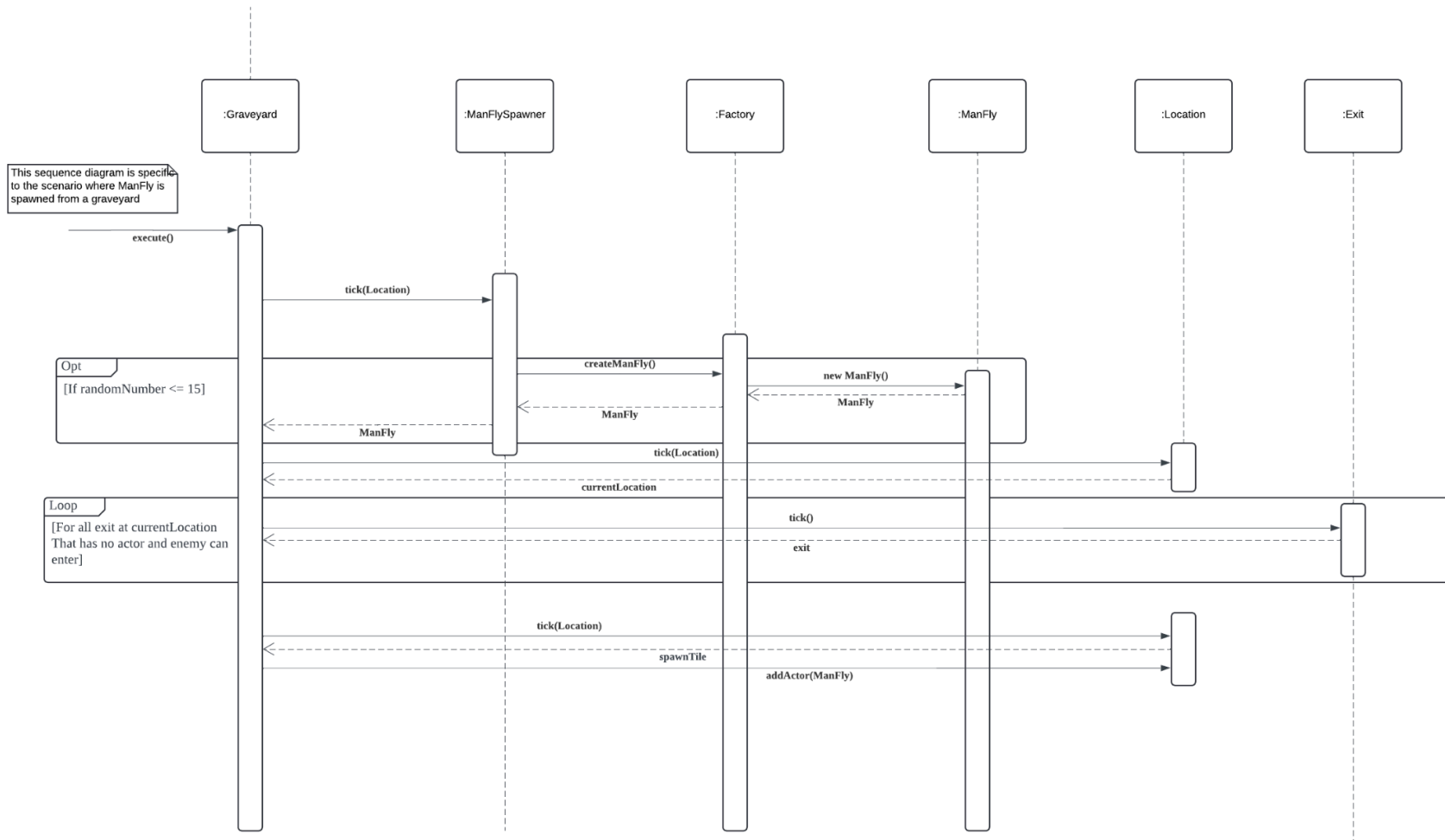
	special capabilities or resistances that an actor may possess. Relationships: 1. The POISON_RES ability specifically grants actors immunity to poison effects inflicted by hazardous terrains like PoisonSwamp. 2. Used by the PoisonSwamp and PoisonEffect classes to check if an actor is resistant to poison before applying any negative effects. 3. Allows actors with POISON_RES to traverse hazardous environments without taking poison damage, enhancing strategic gameplay.	PoisonEffect, and the Actor class. If new resistances or status effects are added, changes may need to be made across multiple classes, which can limit flexibility and increase maintenance effort. Teleportation Configuration: 1. Setting up teleport links using Gate requires careful configuration to ensure that connections between GameMap instances are accurate. Adding multiple destinations and managing teleportation points adds complexity, increasing the risk of misconfiguration or errors during gameplay. Dependency on Visual Feedback: 1. The reliance on FancyMessage for displaying transitions and key events introduces a dependency that could affect the player's experience. Missing or incorrect messages could confuse players, especially during teleportation between areas, which could reduce immersion if not handled correctly.
FancyMessage	FancyMessage is a utility class used to provide visual feedback to players. Relationships: 1. It displays messages such as transitions between different areas, specifically indicating new map names when a player teleports using a Gate. 2. Integrated with TeleportAction to print the destination map's name, helping to maintain player immersion and	

	orientation within the game world. 3. Enhances the narrative element of gameplay by providing meaningful, context-specific messages that enrich the player's exploration experience.	
--	--	--

REQ3: Ailment



Req 3 UML Diagram



Req 3 Sequence Diagram

Classes Modified/Created	Roles and Responsibilities	Rationale
Spawner	Spawner is an interface representing the Spawner for generating entities. Relationships: • It implements Spawner • It depends on Enemy	Alternative Solution: Instead of creating spawners, the Enemies could have handled all the chance rolls and spawn themselves at a given spawner location with SpawnAction. That would mean there is no Spawner interface and instead a SpawnAction class to handle all the code for spawning Finalised Solution: 1. Created a Spawner interface to be used as an attribute in Graveyard class 2. Made specialised spawners to run all the chance and creation of the enemy object 3. Created Factory class to organise and create all enemies through a more tidy manner
ManFly (extends Enemies)	Manfly is a class used to initialise the Manfly and its stats. Relationships: • Extends Enemy: Inherits from the Enemy class, making it a type of enemy in the game. • Depends on WanderBehaviour: ManFly can wander randomly when no target is in range. • Depends on FollowBehaviour: ManFly follows actors that are hostile to it when nearby. • Depends on AttackBehaviour: ManFly prioritizes attacking hostile actors when within range. • Depends on ManFlySting: Its intrinsic weapon allows it to attack using a sting that can apply poison.	

	• Depends on Poison: Applies poison effects on its target during an attack. • Depends on AttackAction: ManFly performs attack actions when engaged with a hostile actor. • Depends on Status: Checks for hostility between the ManFly and other actors.	The SOLID principles—Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion—serve as fundamental guidelines in software design. These principles play a crucial role in enhancing maintainability, flexibility, and clarity in code. The following sections outline how these principles are applied within the game, illustrating their importance in crafting robust software. 1. Single Responsibility Principle (SRP) The Single Responsibility Principle states that a class should have only one reason to change, emphasizing the need for focused responsibilities. In this codebase: • Enemy Classes: Each enemy type, such as Spirit, ManFly, and Scarab, is encapsulated in its own class. Each class manages its unique behaviors and attributes, including attack methods and interactions. • Weapon Classes: Classes like ManFlySting and SpiritHand are dedicated to handling specific weapon mechanics and damage calculations. By adhering to SRP, the code ensures simpler debugging and more manageable modifications. For instance, adding a new attack method to the Spirit class can be accomplished without affecting other enemy classes. 2. Open/Closed Principle (OCP) The Open/Closed Principle posits that software entities should be open for extension but closed for modification. In this context:
Spirit (extends Enemies)	Spirit is a class used to initialise the Spirit along with its stats. Relationships • Extends Enemy: Inherits from the Enemy class, making it a specific type of enemy in the game. • Depends on WanderBehaviour: The Spirit can wander randomly across the map when no target is nearby. • Depends on AttackBehaviour:	

	The Spirit will attack any actor hostile towards it. • Depends on SpiritHand: The Spirit uses its intrinsic weapon, SpiritHand, to attack enemies.	• Spawner Interface: The Spawner interface allows for the addition of new enemy types, such as SpiritSpawner and ManFlySpawner, without requiring changes to existing code. This promotes extensibility while safeguarding existing functionality. • Factory Class: The Factory class centralizes the instantiation logic for enemy objects. New enemy types can be integrated effortlessly without altering existing factory methods.
ManFlySting (extends IntrinsicWeapon)	The ManFlySting class represents the intrinsic weapon used by the Man Fly enemy in the game. This weapon simulates a sting attack, delivering damage to a target while also introducing the potential for a poison effect. Relationships: • It extends IntrinsicWeapon to inherit basic weapon properties and methods. • It depends on Poison to apply poison effects to the target when the sting attack is successful. • It depends on Actor to perform actions on the attacker and target during the attack. • It depends on GameMap to provide context about the game environment during the attack process.	This principle facilitates ongoing development and feature expansion within the game, allowing developers to add new elements without significant codebase alterations. 3. Liskov Substitution Principle (LSP) The Liskov Substitution Principle emphasizes that derived classes should be substitutable for their base class without altering the correctness of the program. In the code: • Enemy Hierarchy: Classes like ManFly and Spirit extend the Enemy base class. Methods operating on an Enemy can seamlessly use either subclass. By following LSP, the game can incorporate various enemy types without introducing new bugs, ensuring a stable gameplay experience. 4. Interface Segregation Principle (ISP) The Interface Segregation Principle states that clients should not be forced to depend on interfaces they do not use. In this codebase: • Minimalist Spawner Interface: The Spawner interface focuses solely on the spawnEnemy() method, preventing clients from being burdened with unnecessary methods.

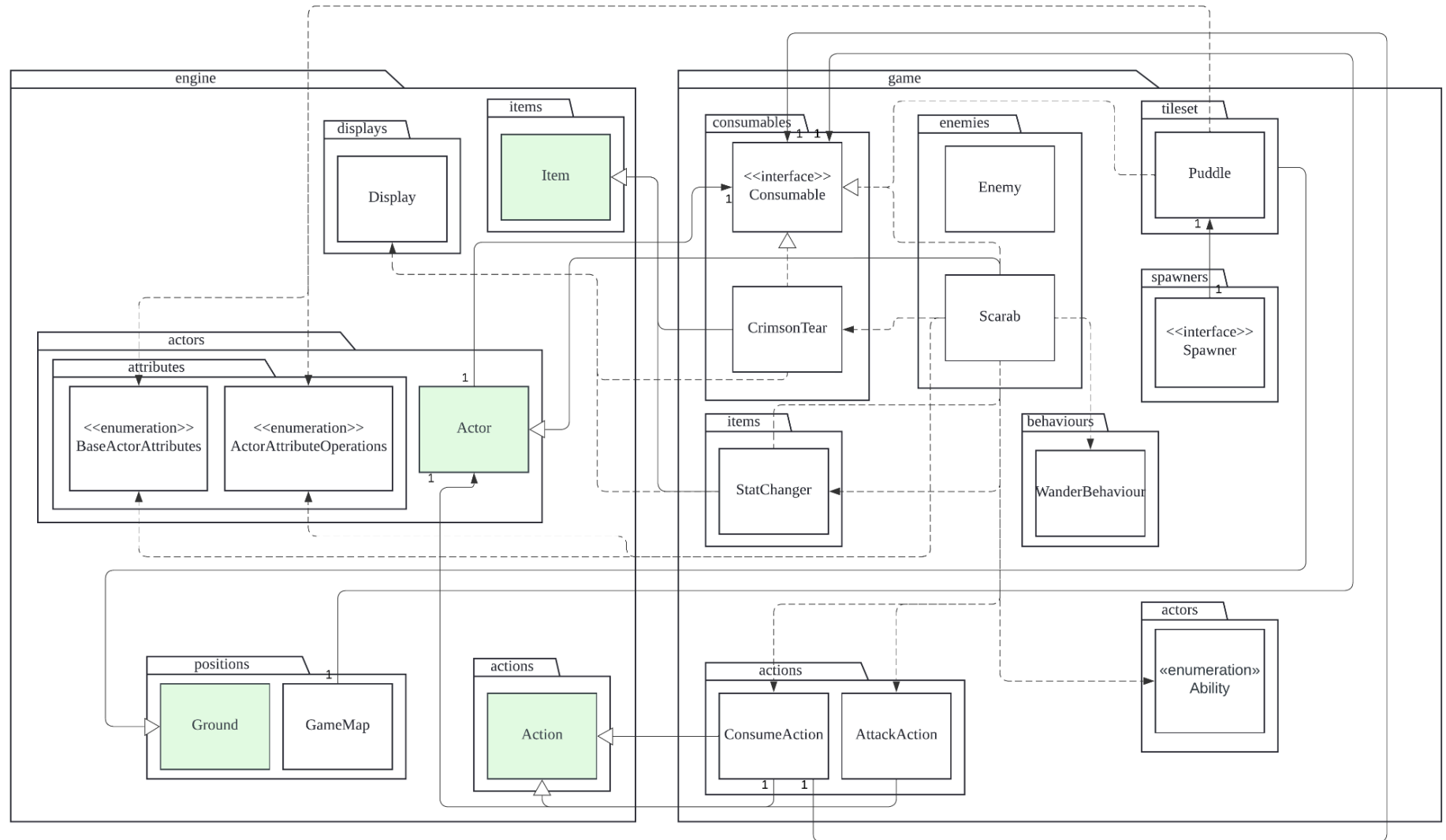
	It checks for the Ability capability (POISON_RES) to determine if the target is immune to poison.	This principle enhances code clarity and usability, allowing developers to understand the functionality of each interface without encountering extraneous concerns.
SpiritHand (extends IntrinsicWeapon)	SpiritHand is a class used to instantiate the Spirit's weapon and its stats. Relationship Extends IntrinsicWeapon: The SpiritHand class is a specialized weapon used by the Spirit enemy, inheriting its properties from the IntrinsicWeapon class.	5. Dependency Inversion Principle (DIP) The Dependency Inversion Principle asserts that high-level modules should not depend on low-level modules but rather on abstractions. In this implementation: Use of Interfaces: High-level game logic components depend on the Spawner interface instead of concrete enemy classes. This abstraction allows for the flexibility to swap out enemy spawners without modifying the underlying game logic. This principle promotes a decoupled architecture, making the codebase more adaptable to changes. For example, introducing a new enemy type or spawner logic can occur without affecting existing systems that utilize the interface.
ManFlySpawner (implements Spawner)	ManFlySpawner is a class responsible for spawning the ManFlies. Relationships: - Implements Spawner: The ManFlySpawner class follows the Spawner interface contract, responsible for spawning ManFly enemies in the game. - Depends on Factory.createManFly(): ManFlySpawner uses a factory to create instances of ManFly enemies.	The implementation of the SOLID principles within this game's codebase not only fosters a robust and maintainable structure but also promotes extensibility and adaptability in design. Ensuring that classes have well-defined responsibilities, allowing for easy extensions, and maintaining clear dependencies leads to a more organized, efficient, and resilient codebase. Limitations and Tradeoffs: Repetition of code: Each Spawner will have an almost identical spawn chance check which could have been eliminated if used an abstract class that can encompass all spawners

	• Depends on RandomNumberGenerator: Utilizes random number generation to determine if a ManFly enemy will spawn, based on a predefined probability.	• Hard to add new Spawners: If a new spawner were to be added then a whole new class would have to be made and they would have to implement Spawner and re-code the entire spawnEnemy() method. Other than that they would have to add the enemy into the factory class to make sure all code runs the same
SpiritSpawner (implements Spawner)	SpiritSpawner is a class responsible for spawning the Spirits. Relationships: • Implements Spawner: The SpiritSpawner class implements the Spawner interface, responsible for spawning Spirit enemies. • Depends on Factory.createSpirit(): Uses the factory to create Spirit enemy instances. • Depends on RandomNumberGenerator: Utilizes random number generation to determine the chance of spawning a Spirit enemy.	

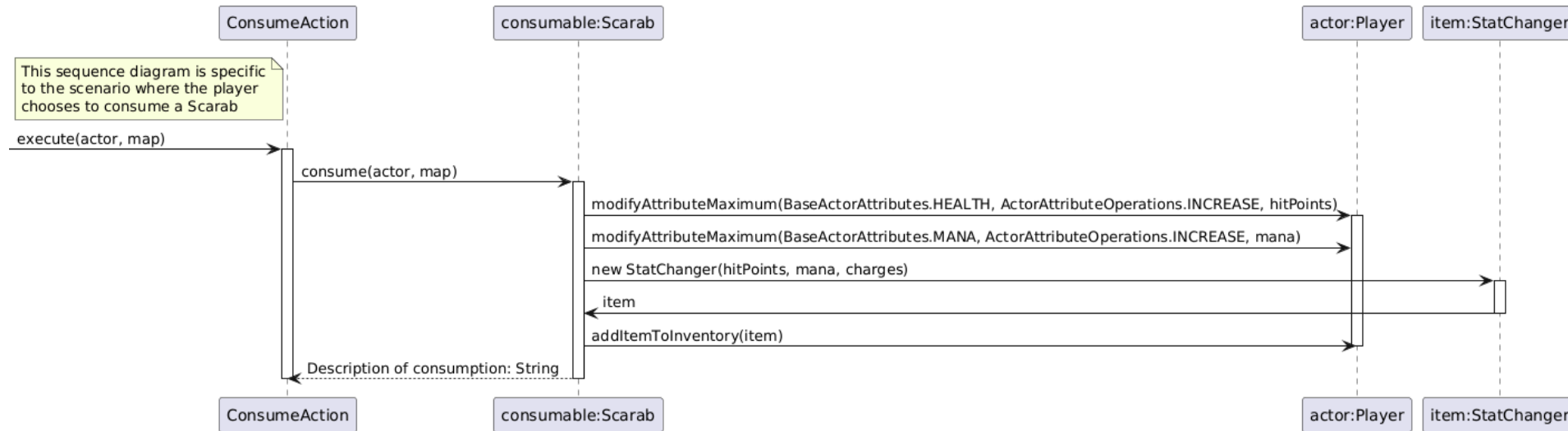
Factory	The Factory class provides centralized methods for creating instances of different enemy types (Spirit, ManFly, Scarab). Relationships: • Depends on Spirit class, ManFly class, and Scarab class: The Factory class is responsible for creating instances of these enemy types and returning them to be used in the game.	
Graveyard (extends Ground)	The Graveyard is responsible for periodically spawning enemies using the provided Spawner. Relationships: • Extends Ground: The Graveyard class is a specific type of Ground in the game world. • Depends on Spawner: The Graveyard uses a Spawner instance to generate enemies periodically. • Interacts with Enemy: It spawns instances of the Enemy class in the game.	

RandomNumberGenerator	The RandomNumberGenerator class provides static utility methods for random number generation, aiding various game functionalities.	
-----------------------	--	--

REQ4: Be wary of dung (beetle)



Req 4 UML Diagram



Req 4 Sequence Diagram

Classes Modified/Created	Roles and Responsibilities	Rationale
Scarab	Class representing the new Scarab NPC Relationships: - It extends Enemy - It implements Consumable in order to perform Consumable actions - It depends on WanderBehaviour in order to perform random wandering - It depends on Ability in order to add PoisonResistance to its capabilities - It depends on CrimsonTear in order to drop this item upon	Alternative: Change Consumable definition for consume without a default method to allow for continued use of the old definition Violates: S - Previously the consume method was only concerned about the interactions between the Actor and the consumable. But this change would require all consumables to also handle Location O - Requires modifications of older Consumables I - Now all classes require handling consume with Location data regardless of whether it is necessary for that class Finalised solution: Create a new default method in Consumable that calls the old consume definition. Change ConsumeAction to rely on the new

	death • It depends on ConsumeAction and AttackAction • It depends on BaseActorAttributes, ActorAttributeOperations and StatChanger in order to modify the Actor's stats upon consumption	consume that passes the GameMap as a parameter (while this is a violation of OCP I believe that it is justified as it allows for easier extensibility in the future). Create a hidden StatChanger item that manages lifetimes non items i.e. when the Player consumes a Scarab the stat changer item is added to the players inventory, when the effect of consuming the scarab is supposed to wear out the StatChanger item in the player's inventory manages that then proceeds to delete itself
CrimsonTear	Class representing the Crimson Tear item Relationships: • It extends Item • It implements Consumable • It depends on Display to display a message to the player when finished.	Reason for decision: The creation of StatChanger ensures that the responsibility of temporary stat management of non items is isolated. Consumable non items only need to worry about applying the effect and don't need to worry about effect wearing off mechanics. (<i>Single Responsibility Principle</i>) The StatChanger can be easily extended in the future in order to allow for other non items that want to temporarily buff the player's stats without modifying existing consumables. (<i>Open Closed Principle</i>)
StatChanger	Class representing a hidden item that handles limited time stat changes Relationships: • It extends Item • It depends on BaseActorAttributes and ActorAttributeOperation to reset the actor's stats back after the effect has worn off • It depends on Display to display a message to the player when it performs its duty.	By using a default method existing consumables can still be used without them having to worry about the GameMap data being passed into the consume method. Any consumables can still be substituted out without introducing unexpected behaviour. (<i>Liskov substitution Principle</i>) The StatChanger item is designed specifically to deal with temporary stat changes and self deletion. This helps the implementation adhere to ISP by not forcing other classes like the Player to handle the temporary stat management. (<i>Interface Segregation Principle</i>)
Puddle	Class representing the puddle type ground Relationships: • It extends Ground	None of my created classes directly depend on concrete classes and they instead depend on abstractions. (<i>Dependency Inversion Principle</i>)

	• It implements Consumable • It has a 1:1 association with Spawner • It has a dependency on Enemy, RandomNumberGenerator	Limitations and tradeoffs: I had to modify the interface Consumable and the ConsumeAction. While the modification of Consumable with a default interface for the new changes does not require any changes in the previously implemented Consumables it still required some changes in the ConsumeAction to call the new method instead of the old one. As previously mentioned this is a violation of OCP but I believe that it is justified as it improves future extensibility by allowing for future Consumables that interact with the GameMap during consumption. The StatChanger is also limited in scope to only managing maximum attribute value changes that expire. CrimsonTear does its own handling on increasing health over time.
Consumable	Interface representing Consumable objects Relationships: • 1:1 association with engine.actors.Actor • 1:1 association with engine.positions.GameMap	
ConsumeAction	Class representing an action to consume Relationships: • 1:1 association with Consumable	